"Security is not a phase, it's a mindset and a continuous responsibility."

MODULE 40

Application Security Testing

Learn how to identify, analyze, and remediate security vulnerabilities in applications using industry-leading security testing practices and tools.

TEKsystems Global Services

WORKFORCE DEVELOPMENT

TEKsystems Global Services

WORKFORCE DEVELOPMENT

MODULE 40 OVERVIEW

Learn how to identify, analyze, and remediate security vulnerabilities in applications using industry-leading security testing practices and tools.

This module covers the full application security testing lifecycle -- OWASP Top 10, SAST, DAST, risk-based triage, secure coding, and CI/CD security gates -- anchored in the Northstar CRM's real security-gate lab.

DURATION & LAB

1 Hour Theory

OWASP Top 10, SAST, DAST, risk assessment, secure coding, and CI/CD security gates.

2 Hours Lab

lab40-crm: OWASP Dependency-Check (SCA), focused manual SAST, and one verified authorization fix -- ~45 min timed path in class, full path 3-4 hrs.

Scenario

Northstar CRM release gate before Docker (Lab 41): no ship on raw scanner volume, no unowned Critical/High findings.

MODULE ARC

Understand the Threats

OWASP Top 10, common vulnerabilities, and where security fits across the SDLC and DevOps pipeline.

Test and Assess

Apply SAST and DAST, then interpret findings with severity, risk, and false-positive judgment.

Secure the Pipeline

Apply secure coding practices, enforce CI/CD security gates -- then prove it in lab40-crm.

KEY TAKEAWAY Total time: 3 hours (1 hour theory + 2 hours lab; a ~45-minute timed path is used in class, with the full path available for 3-4 hours of extended practice). By the end, you will find, assess, and fix real vulnerabilities -- not just count them.

WHY SECURITY TESTING MATTERS

Security testing helps identify vulnerabilities early, protects users and data, and ensures your applications are secure, resilient, and trustworthy.

WHY IT MATTERS

Protects Against Real Threats

Attackers constantly look for weaknesses -- testing stops them before they exploit.

Safeguards Sensitive Data

Protects customer, business, and financial data from breaches and unauthorized access.

Ensures Business Continuity

Prevents vulnerabilities that could cause downtime, data loss, or service disruption.

Meets Compliance

Helps meet security standards and regulations (GDPR, PCI-DSS, ISO 27001).

Builds Trust

Secure applications build customer trust and protect your brand's reputation.

BUSINESS OUTCOMES

- **Prevent Security Breaches**
- **Protect Users & Data**
- **Reduce Risk & Costs**
- **Increase Quality & Reliability**
- **Strengthen Business Confidence**
- **Ensure Compliance & Standards**

WHAT SECURITY TESTING IS NOT

It is not a one-time checkbox exercise, and it is not solely the tester's job -- it is a continuous, shared responsibility across development, security, and operations.

KEY TAKEAWAY Security testing is not just about finding bugs -- it's about building secure, reliable, and trusted applications that protect your business and your customers.

WHY SECURITY TESTING MATTERS

IDENTIFIES VULNERABILITIES EARLY
Find and fix security flaws before attackers do.

REDUCES RISK
Minimizes the chance of security breaches and data loss.

SAVES TIME & COST
Fixing issues early is far cheaper than after a breach.

IMPROVES QUALITY
Builds secure, reliable, and trustworthy applications.

PROTECTS DATA & PRIVACY
Safeguards sensitive data and user privacy.

ENSURES COMPLIANCE
Meets regulatory and industry security standards.

BUILDS CUSTOMER TRUST
Demonstrates commitment to security and earns user confidence.

SUPPORTS BUSINESS CONTINUITY
Prevents downtime and protects brand reputation.

SECURE APPLICATIONS. STRONGER BUSINESS.

Fewer Breaches | Lower Costs | Happier Users | Sustainable Growth

MODULE LEARNING OBJECTIVES

By the end of this module, you will be able to find, test, assess, and fix application security vulnerabilities using SAST, DAST, and secure DevOps practices.

- **1. Understand Security Fundamentals —** explain the importance of security testing and key principles of secure software development.
- **2. Identify Common Vulnerabilities —** recognize OWASP Top 10 vulnerabilities and their impact on applications.
- **3. Apply Static Security Testing (SAST) —** understand how SAST works, the tools used, and interpret findings to identify code-level flaws.
- **4. Apply Dynamic Security Testing (DAST) —** understand how DAST works, the tools used, and identify runtime vulnerabilities in applications.
- **5. Analyze and Prioritize Security Findings —** interpret security reports, classify vulnerability severity, and assess risk accurately.
- **6. Implement Secure Practices in DevOps —** integrate security testing into CI/CD pipelines and enforce security gates before production.

KEY TAKEAWAY These objectives will help you build the skills and knowledge to perform effective application security testing and deliver secure, reliable, and resilient applications.

SECURING A CUSTOMER MANAGEMENT PLATFORM

Enterprise scenario: your organization is building a Customer Management Platform (CMP) used by thousands of customers and internal teams to manage personal data, accounts, and transactions.

KEY BUSINESS CONTEXT

- Stores sensitive customer data (PII, contact details, account info).
- Handles financial transactions and support interactions.
- Exposed to web and mobile users via public APIs.
- Integrated with payment gateway, email service, and third-party systems.

WHY SECURITY TESTING MATTERS HERE

- Protect customer trust and brand reputation.
- Prevent data breaches and financial fraud.
- Meet regulatory and compliance requirements (GDPR, PCI-DSS, ISO 27001).
- Ensure business continuity and minimize downtime.

WHAT CAN GO WRONG WITHOUT SECURITY TESTING

Data Breach

Attackers steal customer PII and sensitive data.

Financial Loss

Unauthorized transactions and fraudulent activities.

System Compromise

Attackers exploit vulnerabilities to take control of the system.

Compliance Penalties

Failing security standards leads to fines and legal issues.

Reputation Damage

Loss of customer trust and long-term impact on business.

OUR SECURITY TESTING APPROACH

1. **Identify** — Assets, entry points, threats.
2. **Test** — Perform SAST early.
3. **Validate** — Use DAST at runtime.
4. **Analyze** — Prioritize risk & impact.
5. **Remediate** — Fix and verify the fixes.

KEY TAKEAWAY Security testing is not optional -- it's essential to protect customers, data, and business. Use the right tools, processes, and best practices to build secure applications from the start.

ENTERPRISE SCENARIO: SECURING A CUSTOMER MANAGEMENT PLATFORM

CUSTOMER JOURNEY

1. Register / Login
Customer creates account or logs in.

2. Manage Profile
Customer updates profile and preferences.

3. Interact & Transact
Customer browses, places orders, and requests support.

4. Get Support
Customer raises tickets and gets assistance.

5. Logout
Customer logs out securely.

SECURE CUSTOMER MANAGEMENT PLATFORM

CLIENTS
- Web (React)
- Mobile (iOS/Android)
- Third-Party Integrations

EDGE & SECURITY LAYER
- WAF
- DDoS Protection
- SSL/TLS Encryption
- Rate Limiting & Bot Protection

APPLICATION LAYER
- API Gateway
- Auth Service (OAuth 2.0 / OIDC)
- Customer Service
- Order Service
- Support Service

DATA LAYER
- Operational DB (PostgreSQL)
- Cache (Redis)
- File Storage (S3 / Object Store)
- Analytics DB (Data Warehouse)

INTEGRATIONS
- Email Service
- Payment Gateway
- SMS / Notification
- CRM / ERP Systems

PLATFORM SECURITY FOUNDATION
- IAM & RBAC
- Secrets Management
- Encryption at Rest
- Audit Logging & Monitoring
- Vulnerability Management
- Backup & Disaster Recovery
- Secure SDLC (DevSecOps)

SECURITY CONTROLS (KEY)

Strong Authentication
MFA, Password Policy, Session Management

Authorization
RBAC, Least Privilege Access

Input Validation
Prevent Injection, XSS, CSRF

Data Protection
Encryption in Transit & at Rest, Tokenization

Secure APIs
API Security, Schema Validation, Throttling

Monitoring & Detection
SIEM, Alerts, Anomaly Detection

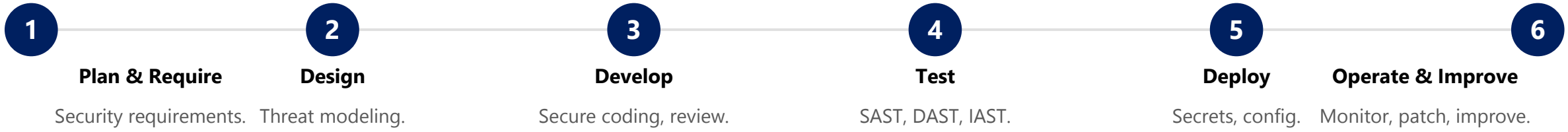
Incident Response
Playbooks, Forensics, Continuous Improvement

BUSINESS OUTCOMES

- Protects Customer Data & Privacy
- Builds Customer Trust & Loyalty
- Ensures Compliance (GDPR, PCI DSS, etc.)
- Reduces Risk & Downtime
- Enables Secure Business Growth

SECURE SOFTWARE DEVELOPMENT LIFECYCLE (SSDLC)

The SSDLC integrates security practices into every phase of the software development process to build secure, reliable, and resilient applications.



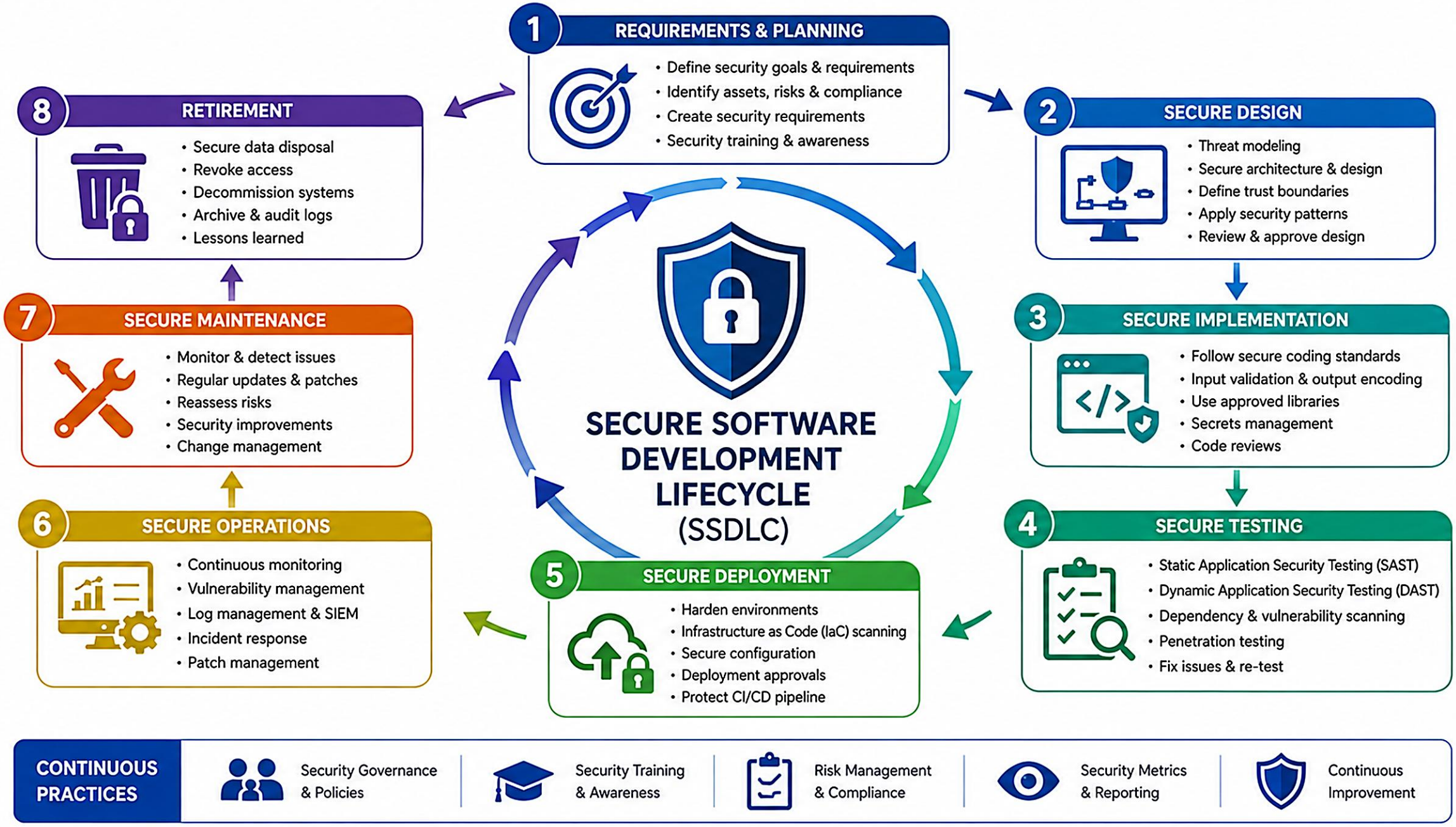
SSDLC PRINCIPLES

- **Proactive —** fix issues early, not late.
- **Everyone's Responsibility —** security is a team accountability.
- **Defense in Depth —** use multiple layers of security.
- **Compliance by Design —** meet standards from the start.
- **Continuous Improvement —** evolve with new threats.

BUSINESS OUTCOMES

- Reduce risk of breaches.
- Build customer trust.
- Ensure regulatory compliance.
- Lower cost of fixes -- issues found early cost far less to fix.
- Improve quality and reliability.

KEY TAKEAWAY SSDLC ensures security is integrated into every phase -- resulting in secure applications, protected data, and trusted experiences.



COMMON APPLICATION VULNERABILITIES

Application vulnerabilities are weaknesses in software that attackers can exploit to gain unauthorized access, steal data, or disrupt systems.

#	Vulnerability	Description	Potential Impact
1	Injection	Untrusted input sent to an interpreter as part of a command or query.	Data breach, unauthorized access, data manipulation
2	Broken Authentication	Flaws in authentication mechanisms let attackers compromise user accounts.	Account takeover, privilege escalation, identity theft
3	Broken Access Control	Applications fail to properly restrict access to resources and functionality.	Unauthorized data access, privilege escalation, business logic abuse
4	Sensitive Data Exposure	Applications expose sensitive data due to insufficient protection.	Data breach, compliance violations, loss of customer trust
5	Cross-Site Scripting (XSS)	Attackers inject malicious scripts into web pages viewed by other users.	Session hijacking, phishing attacks, defacement
6	Security Misconfiguration	Improper configuration of servers, frameworks, or cloud services.	Unauthorized access, data leakage, system compromise
7	Using Components with Known Vulnerabilities	Using outdated or vulnerable third-party libraries and components.	System compromise, data breach, loss of availability

HOW TO REDUCE THESE RISKS

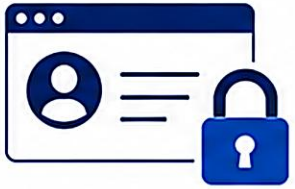
Follow Secure Coding Practices | **Validate & Sanitize Input** | **Strong AuthN & Access Control** | **Protect Sensitive Data**

Keep Components Up To Date | **Continuously Test & Monitor**

KEY TAKEAWAY Understanding these common vulnerabilities helps development and security teams build secure applications and protect business and customer data.

COMMON APPLICATION VULNERABILITIES

1 BROKEN ACCESS CONTROL



Attackers gain unauthorized access to sensitive data or functionality.

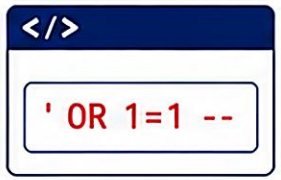
Example: Accessing another user's account or admin panel.

2 CRYPTOGRAPHIC FAILURES

Sensitive data is not properly protected during storage or transmission.

Example: Storing passwords in plain text.

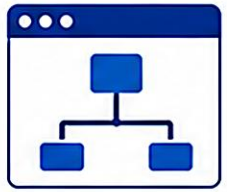
3 INJECTION



Untrusted input is sent to an interpreter as part of a command or query.

Example: SQL Injection, NoSQL Injection, Command Injection.

4 INSECURE DESIGN



Security flaws due to missing or ineffective security controls in design.

Example: Missing threat modeling or security requirements.

5 SECURITY MISCONFIGURATION

Improper configuration exposes systems and data to attackers.

Example: Default credentials or open cloud storage.

6 VULNERABLE AND OUTDATED COMPONENTS

Using components with known vulnerabilities can be exploited by attackers.

Example: Outdated libraries or plugins.

7 IDENTIFICATION AND AUTHENTICATION FAILURES

Weak or missing authentication allows attackers to compromise accounts.

Example: Weak passwords or lack of MFA.

8 SOFTWARE AND DATA INTEGRITY FAILURES

Untrusted data or code is accepted without proper validation.

Example: Unsigned updates or unsafe deserialization.

9 SECURITY LOGGING AND MONITORING FAILURES

Insufficient logging and monitoring hinder detection and response to attacks.

Example: Missing logs for critical events.

10 SERVER-SIDE REQUEST FORGERY (SSRF)

Attackers can make a server send requests to unintended internal or external resources.

Example: Accessing internal services via vulnerable endpoint.

Secure coding, proper validation, least privilege, and continuous monitoring help prevent these vulnerabilities.

OWASP TOP 10 OVERVIEW

The OWASP Top 10 is a standard awareness document representing the most critical security risks to web applications.

#	Risk	Description	Key Prevention
01	Injection	Malicious code (SQL, NoSQL, OS commands, LDAP) injected into an application.	Parameterized queries, input validation, least privilege
02	Broken Authentication	Flaws allow attackers to compromise user accounts.	Strong password policy, MFA, secure session management
03	Broken Access Control	Applications fail to properly enforce access controls on resources.	Enforce least privilege, test access control thoroughly
04	Insecure Design	Security flaws in the design phase lead to vulnerable architecture.	Threat modeling, secure design principles, security review
05	Security Misconfiguration	Improper configuration of servers, frameworks, or cloud services.	Harden systems, remove unnecessary features, review configs
06	Vulnerable and Outdated Components	Using components with known vulnerabilities that attackers can exploit.	Keep components updated, use dependency-scanning tools
07	Cross-Site Scripting (XSS)	Malicious scripts injected into web pages viewed by other users.	Output encoding, input validation, Content Security Policy
08	Insecure Deserialization	Untrusted data is deserialized, leading to code execution or other attacks.	Avoid deserializing untrusted data, use safe formats
09	Using Components with Known Vulnerabilities	Third-party components with publicly known vulnerabilities are exploited.	Monitor vulnerability databases, use SCA tools
10	Insufficient Logging & Monitoring	Failure to log and monitor prevents detection of and response to attacks.	Log important events, monitor and alert, retain logs securely

KEY TAKEAWAY Addressing the OWASP Top 10 risks helps organizations reduce their attack surface, protect data, and build trust with customers. Threats evolve, so keep learning, testing, and improving continuously.

OWASP TOP 10 OVERVIEW

OWASP
Secure applications.
Protect the world.

The most critical web application security risks

1 BROKEN ACCESS CONTROL

Attackers can access unauthorized data or perform actions outside their permissions.

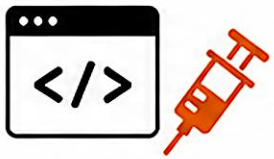
Example: Accessing another user's account or admin functions.

2 CRYPTOGRAPHIC FAILURES

Sensitive data is not adequately protected in transit or at rest.

Example: Storing passwords in plain text.

3 INJECTION



Untrusted input is sent to an interpreter as part of a command or query.

Example: SQL Injection, NoSQL Injection, OS Command Injection.

4 INSECURE DESIGN

Security flaws due to missing or ineffective security controls in design.

Example: Missing threat modeling or secure design principles.

5 SECURITY MISCONFIGURATION

Improper configuration exposes systems and data to attackers.

Example: Default credentials or open cloud storage.

6 VULNERABLE AND OUTDATED COMPONENTS

Using components with known vulnerabilities can be exploited by attackers.

Example: Outdated libraries or plugins.

7 IDENTIFICATION AND AUTHENTICATION FAILURES

Weak or missing authentication allows attackers to compromise accounts.

Example: Weak passwords or lack of MFA.

8 SOFTWARE AND DATA INTEGRITY FAILURES

Untrusted data or code is accepted without proper validation.

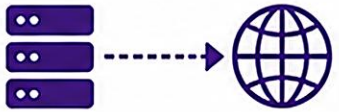
Example: Unsigned updates or unsafe deserialization.

9 SECURITY LOGGING AND MONITORING FAILURES

Insufficient logging and monitoring hinder detection and response to attacks.

Example: Missing logs for critical events.

10 SERVER-SIDE REQUEST FORGERY (SSRF)



Attackers can make a server send requests to unintended internal or external resources.

Example: Accessing internal services via vulnerable endpoint.

Secure by Design. Validate Everything. Keep Systems Updated. Monitor Continuously.

Reduce Risk | Protect Data | Build Trust | Ensure Resilience

TRY IT YOURSELF — EXERCISE 1: MAP CRM ATTACK SURFACES

Reinforces: mapping the OWASP Top 10 themes you just saw to real Northstar CRM attack surfaces, before we look at how DevOps pipelines test for them.

STEPS (notes/lab40-owasp-surface-map.md)

Step	What To Do
1 — Inventory Touchpoints	List at least five CRM surfaces: HTTP APIs, JWT/RBAC, SQL/JPA, file/log sinks, and (later) Kafka. Mark which hold PII vs. IDs.
2 — Check the Reference	Compare your list to OWASP themes: injection, broken access control, security misconfiguration, vulnerable components, logging failures.
3 — Rank Top Three	Pick the three highest-risk surfaces for a release gate before containers; write one sentence of business impact per item.
4 — Anchor to Fixtures	Use Amina (CUS-1001, ACTIVE) and Ravi (CUS-1002, PROSPECT) as the agents looking up customer records in every example.

Reference surface map (fill in)

```
| Surface                | OWASP theme                | Example              |
| Customer GET/PUT API   | Broken access control      | Agent reads CUS-1001 |
| Search query params    | Injection                  | Name/email filters   |
| pom.xml dependencies   | Vulnerable components      | Transitive CVE       |
| application.yml secrets | Security misconfiguration | DB password in Git   |
```

KEY TAKEAWAY TRY IT NOW — Complete Exercise 1 before continuing: exercises/exercise-01-owasp-surface-map.md (workspace: examples/module-40-exercises).

SECURITY TESTING IN MODERN DEVOPS

In modern DevOps, security is integrated into every stage of the delivery pipeline. It's continuous, automated, and everyone's responsibility.

1. **Plan** — Threat modeling.
2. **Code** — Secure coding, SAST.
3. **Build** — SCA, secrets scan.
4. **Test** — DAST, API testing.
5. **Release & Deploy** — Image scan, config.
6. **Operate** — Monitor, respond.

WHY IT MATTERS

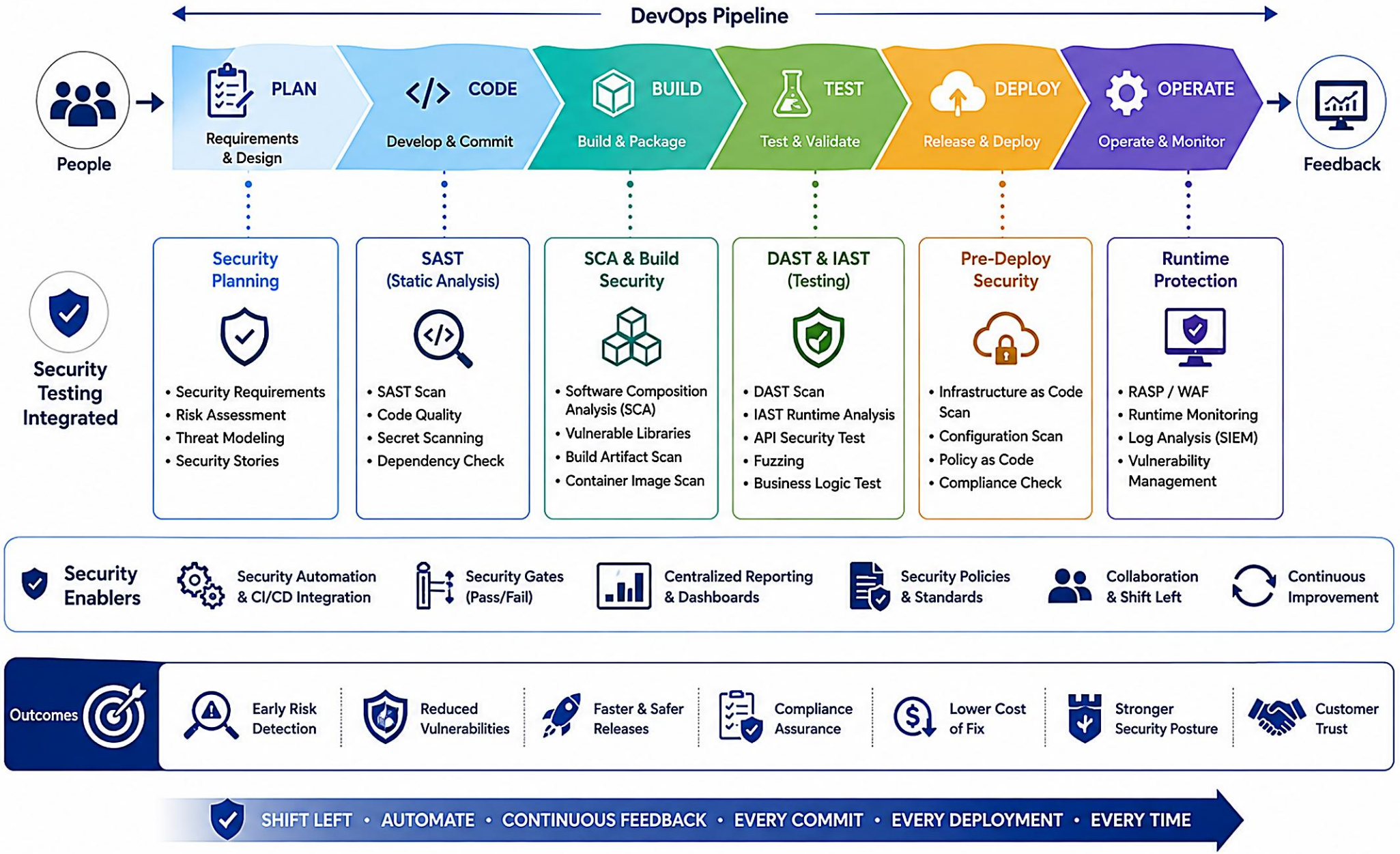
- **Detect vulnerabilities early —** find and fix issues before they reach production.
- **Faster, safer releases —** automate security checks without slowing delivery.
- **Reduce risk and cost —** prevent breaches, rework, and compliance penalties.
- **Shared responsibility —** everyone in the pipeline owns security.

SHIFT-LEFT SECURITY: COST TO FIX

Stage	Fix Cost	Timing
Design / Requirements	$0.10	Earliest fix
Development	$1	Early fix
Staging / QA	$10	Late fix
Production	$100	Latest fix

KEY TAKEAWAY Security testing in DevOps is about speed with security. Embed security early, automate continuously, and deliver value securely.

SECURITY TESTING IN MODERN DEVOPS



TRY IT YOURSELF — EXERCISE 2: PLAN DEPENDENCY-CHECK GATE

Reinforces: turning the SCA / dependency-scan pipeline stage you just saw into a concrete Maven security-scan profile for lab40-crm.

STEPS (notes/lab40-dependency-check-plan.md)

Step	What To Do
1 — Profile Sketch	Plan a Maven profile -Psecurity-scan: plugin goal, HTML+JSON report formats, and a CVSS fail-threshold placeholder.
2 — Confirm the Command	./mvnw -B -Psecurity-scan dependency-check:check, run from the CRM module root under JDK 21 + Maven Wrapper.
3 — Draft Suppression Policy	Write the three required fields for any suppression: CVE id, owner, and expiry date -- silent suppressions fail the gate.
4 — Prep Evidence Folders	Create note paths for sanitized HTML/JSON reports under notes/screenshots/lab-40/ (do not run the full scan yet).

security-scan Maven profile (sketch)

```
<profile>
  <id>security-scan</id>
  <build><plugins><plugin>
    <groupId>org.owasp</groupId><artifactId>dependency-check-maven</artifactId>
    <configuration>
      <formats><format>HTML</format><format>JSON</format></formats>
      <failBuildOnCVSS>7</failBuildOnCVSS>
    </configuration>
  </plugin></plugins></build>
</profile>
```

KEY TAKEAWAY TRY IT NOW — Complete Exercise 2 before continuing: exercises/exercise-02-dependency-check-plan.md (workspace: examples/module-40-exercises).

WHAT IS SAST?

SAST (Static Application Security Testing) analyzes source code, bytecode, or binaries without executing the application to find security vulnerabilities early.

HOW IT WORKS

- Scans the codebase (source, bytecode, or binaries).
- Analyzes code using rules, patterns, and data-flow analysis.
- Identifies potential vulnerabilities and weaknesses.
- Generates a report with findings, severity, and remediation guidance.
- Helps developers fix issues early in the SDLC.

WHAT SAST CAN FIND

- SQL Injection
- Cross-Site Scripting (XSS)
- Hardcoded Credentials
- Insecure Cryptographic Usage
- Path Traversal
- Command Injection
- Broken Authentication & Authorization

WHERE SAST FITS (SHIFT-LEFT)

1 **Commit** — 2 **SAST** — 3 **Build** — 4 **Deploy**

Early feedback.

SAST AT A GLANCE

1 **Source Code**
Committed to the repo.

2 **SAST Scanner**
Scans without executing.

3 **Analysis Engine**
Rules, patterns, data flow.

4 **Results & Report**
Findings with severity.

5 **Developer Action**
Review & fix before merge.

KEY TAKEAWAY SAST inspects application code statically to identify security flaws such as injection, authentication issues, and insecure data handling -- before the code runs.

WHAT IS SAST?

SAST (Static Application Security Testing) is a security testing methodology that analyzes source code, bytecode, or binaries without executing the application to find security vulnerabilities early in the SDLC.

HOW SAST WORKS

WHAT SAST DETECTS

- Injection flaws (SQL, Command, etc.)
- Cross-Site Scripting (XSS)
- Insecure authentication & authorization
- Sensitive data exposure
- Insecure configurations
- Code quality & security best practices

KEY BENEFITS

- Finds vulnerabilities early in the SDLC
- Saves time and reduces cost
- Improves code quality & security
- Supports compliance & security standards
- Reduces risk of security breaches

WHERE SAST FITS IN THE SDLC

SAST is performed early (mainly during coding) to shift security left.

HOW SAST WORKS

SAST performs a static analysis of code structure, syntax, data flow, control flow, and security rules -- before the application runs.

1. **Code Input**
2. **Parsing & Analysis**
3. **Rule Matching**
4. **Vuln. Detection**
5. **Prioritization**
6. **Report Generation**
7. **Remediation**

EXAMPLE: SQL INJECTION DETECTED BY SAST

```
String user = request.getParameter("user");
String query = "SELECT * FROM users " +
    "WHERE username = '" + user + "'";
Statement stmt = conn.createStatement();
ResultSet rs = stmt.executeQuery(query);
```

SAST FINDING

- **SQL Injection —** File UserDAO.java, lines 2-5, Severity High, CWE-89.
- **Why —** User input is concatenated into the SQL query, enabling injection attacks.
- **Recommendation —** Use parameterized queries (PreparedStatement).

WHAT SAST ANALYZES

Control Flow | **Data Flow** | **Taint Analysis** | **API / Function Analysis** | **Configuration Analysis**

KEY TAKEAWAY SAST behaves like a meticulous reviewer: it understands code structure and data flow to catch vulnerabilities before runtime, with no need to execute the application.

HOW SAST WORKS

1 SOURCE CODE INPUT

Developer commits code to the repository.

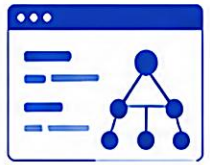
Input: Source code (Java, Python, JS, etc.)

2 SAST SCANNER INITIATED

SAST tool is triggered (manual or in CI/CD pipeline).

The scanner parses and processes the code.

3 CODE PARSING & ANALYSIS



Code is parsed into an Abstract Syntax Tree (AST).

The code structure and logic are analyzed.

4 PATTERN MATCHING

The tool compares code against a database of security rules and vulnerability patterns.

Detects insecure code patterns and practices.

5 VULNERABILITY DETECTION

Potential vulnerabilities are identified with severity and context.

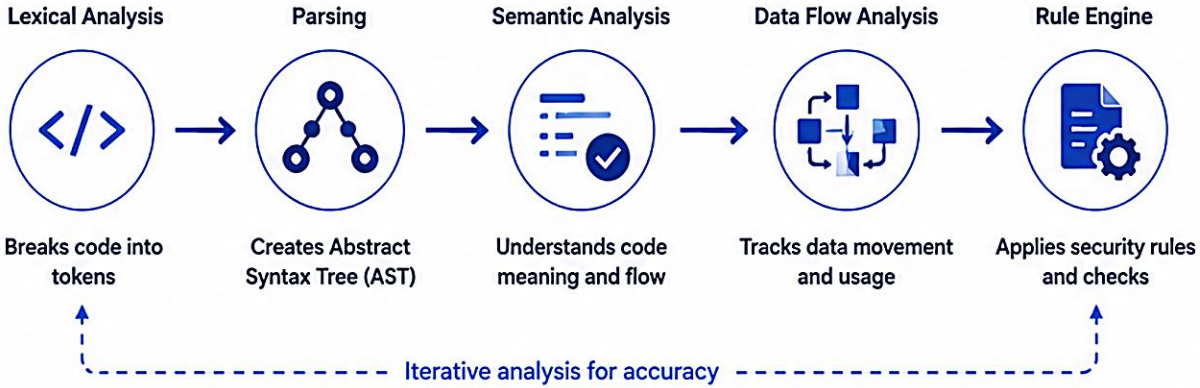
Each issue is mapped to CWE, OWASP, or other standards.

6 REPORTING & REMEDIATION

Detdetailed report is generated with results and guidance.

Developers fix issues and re-run the scan.

INSIDE THE SAST SCANNER



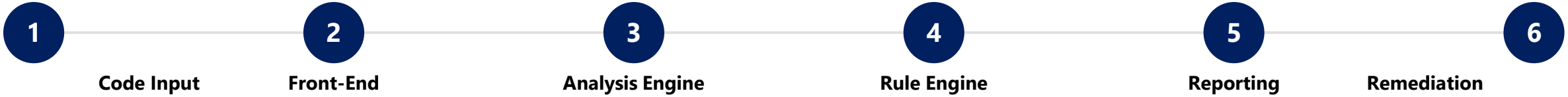
SAST OUTPUT

Comprehensive Security Report

- Vulnerability Name & Description
- File Path & Line Number
- Severity (High / Medium / Low)
- CWE / OWASP Mapping
- Remediation Guidance
- Code Snippet (Context)

SAST ARCHITECTURE

SAST tools analyze source code, bytecode, or binaries without executing the application. Multiple components work together to deliver accurate, actionable findings.



KEY COMPONENTS

- **Code Input —** accepts code from IDE, repositories, or CI/CD.
- **Front-End Processing —** parses code, builds the AST and control-flow graph.
- **Core Analysis Engine —** performs data-flow, control-flow, and taint analysis.
- **Rule Engine —** applies OWASP Top 10, CWE, and custom rules.
- **Results & Reporting —** produces findings with severity and remediation.
- **Feedback & Remediation —** developers fix issues and re-scan.

TYPICAL COVERAGE

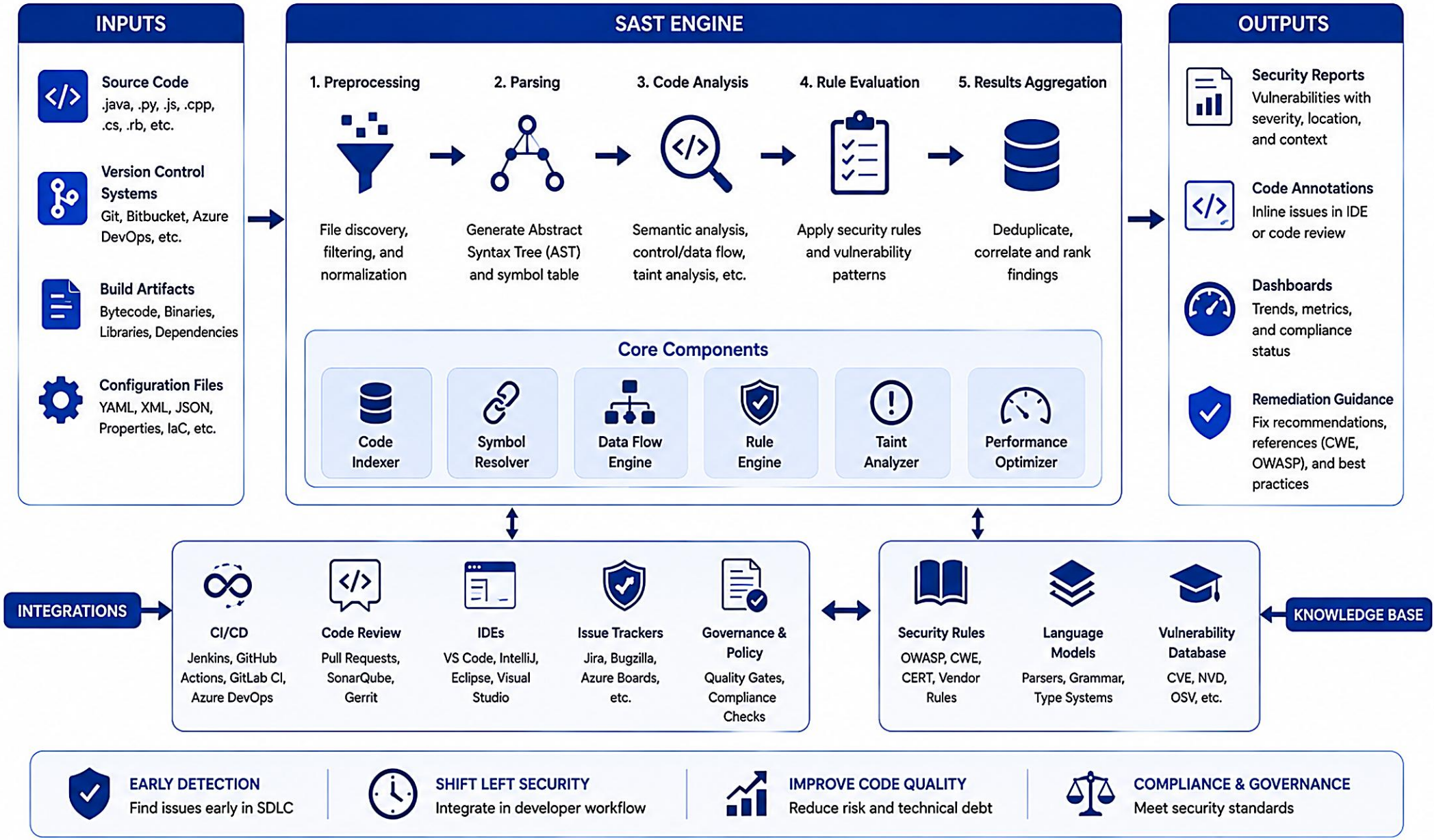
- Injection flaws
- Authentication & authorization
- Data validation & sanitization
- Cryptography issues
- Insecure file handling
- Configuration issues

KEY BENEFITS

- Finds vulnerabilities early in the SDLC
- No need to execute the application
- Automated and scalable
- Supports multiple languages & frameworks
- Reduces risk, cost, and rework

KEY TAKEAWAY SAST is most effective when used early and continuously (shift-left security), alongside other testing types like DAST, for comprehensive coverage.

SAST ARCHITECTURE



COMMON VULNERABILITIES DETECTED BY SAST

SAST tools analyze source code, bytecode, or binaries to find security weaknesses early in the SDLC without executing the application.

#	Vulnerability	Risk	How SAST Detects	Remediation
1	SQL Injection (CWE-89)	High	Detects user input reaching SQL execution sinks unparameterized.	Use PreparedStatement with bound parameters.
2	Cross-Site Scripting (CWE-79)	High	Detects untrusted data in web output (HTML, JS, attributes).	Encode output; escape HTML.
3	Hardcoded Credentials (CWE-798)	High	Detects patterns matching secrets, keys, passwords, tokens.	Read secrets from environment variables/vault.
4	Path Traversal (CWE-22)	High	Detects user input flowing into file APIs without sanitization.	Validate and normalize paths against a base directory.
5	Command Injection (CWE-78)	High	Detects user input in OS command execution functions.	Use ProcessBuilder with fixed arguments.
6	Insecure Deserialization (CWE-502)	High	Detects deserialization of untrusted data sources.	Use allowlist validation or safe formats (JSON+schema).
7	Insecure Logging (CWE-117)	Medium	Detects sensitive data (passwords, tokens, PII) in logs.	Never log sensitive data; mask before logging.
8	Use of Weak Cryptography (CWE-327)	Medium	Detects usage of weak or deprecated algorithms (e.g., MD5).	Use strong algorithms and adequate key sizes.

KEY TAKEAWAY SAST finds issues early -- when they are easiest and least expensive to fix. Integrate SAST early in the SDLC and automate scans in CI/CD pipelines.

COMMON VULNERABILITIES DETECTED BY SAST

1 SQL INJECTION

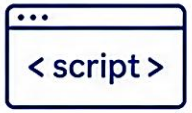
What it is: Untrusted input is used in SQL queries.

Example (Insecure):

```
String query = "SELECT * FROM Users
WHERE id = " + userId;
```

Risk: Data breach, data manipulation, unauthorized access.

2 CROSS-SITE SCRIPTING (XSS)



What it is: Untrusted data is sent to a web page without proper validation.

Example (Insecure):

```
out.println("<div>" + userInput +
"</div>");
```

Risk: Steal cookies, session hijacking, defacement.

3 COMMAND INJECTION

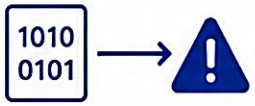
What it is: Untrusted input is used in OS commands.

Example (Insecure):

```
Runtime.getRuntime().exec(
"ping " + userInput);
```

Risk: Remote code execution, system compromise.

4 INSECURE DESERIALIZATION



What it is: Deserializing untrusted data can lead to code execution.

Example (Insecure):

```
ObjectInputStream ois = new
ObjectInputStream(inputStream);
Object obj = ois.readObject();
```

Risk: Remote code execution.

5 HARDCODED CREDENTIALS

What it is: Storing sensitive credentials in source code.

Example (Insecure):

```
String password = "P@ssw0rd123";
```

Risk: Credential leakage, unauthorized access.

6 INSECURE AUTHENTICATION AND AUTHORIZATION

What it is: Weak or missing authentication and improper access control.

Example (Insecure):

```
if(user.isAdmin()) {
    // allow access
}
```

Risk: Unauthorized access, privilege escalation.

7 SENSITIVE DATA EXPOSURE

What it is: Sensitive data is not properly encrypted or is exposed.

Example (Insecure):

```
String cardNumber = request.getParam
("cardNumber");
```

Risk: Data breach, privacy violations.

8 PATH TRAVERSAL

What it is: User input allows access to files outside the intended directory.

Example (Insecure):

```
File file = new File("/var/www/" +
userInput);
```

Risk: Read sensitive files, disclosure of server data.

9 INSECURE CONFIGURATION

What it is: Security misconfigurations in code or framework.

Example (Insecure):

```
// Debug mode enabled in production
app.setDebug(true);
```

Risk: Information leakage, system compromise.

10 USE OF VULNERABLE COMPONENTS

What it is: Using libraries or components with known vulnerabilities.

Example (Insecure):

```
<dependency>
  <artifactId>old-lib</artifactId>
  <version>1.0.0</version>
</dependency>
```

Risk: Exploitable known vulnerabilities, supply chain attacks.

WHY IT MATTERS

- Detects issues early in the SDLC
- Reduces cost of fixing vulnerabilities
- Improves code quality and security
- Supports compliance and standards
- Minimizes risk of data breaches

SAST TOOLS OVERVIEW

SAST tools help organizations find and fix security issues in code early, improve code quality, and support secure delivery at scale.

Tool	Key Features	Best For	Deployment
SonarQube	Static code analysis & quality, security vulnerability detection, custom quality gates.	Dev teams wanting code quality + security in one platform.	Self-hosted or cloud
Checkmarx	Deep static analysis, data-flow & taint analysis, SAST + SCA + IaC.	Large enterprises with complex apps and strict compliance.	Cloud or on-prem
Snyk	Fast SAST with developer focus, fix suggestions & PR integration.	Dev teams, startups, cloud-native applications.	Cloud (Snyk platform)
Veracode	Static analysis as a service, scalable on-demand scanning.	Enterprises needing cloud scale and minimal setup.	Cloud (Veracode platform)
GitHub CodeQL	Semantic code analysis, custom queries, native GitHub integration.	Organizations on GitHub wanting deep, customizable analysis.	GitHub Code Scanning / self-hosted
Qodana (JetBrains)	Static analysis by JetBrains, code quality & security, IDE/CI integration.	Teams using JetBrains IDEs wanting affordable SAST.	Cloud or self-hosted

BEST PRACTICES FOR USING SAST TOOLS

- **Integrate Early (Shift-Left)**
- **Automate Scans in CI/CD**
- **Tune & Suppress False Positives**
- **Fix High-Risk Issues First**
- **Educate Developers**

KEY TAKEAWAY No single tool is perfect. Choose the tool that fits your technology stack, team workflow, compliance needs, and budget.

SAST Tools Overview

COMMERCIAL TOOLS

SonarQube
Multi-language support,
Extensible rules

Checkmarx One
Comprehensive SAST,
DevSecOps integration

Veracode
Cloud-native SAST,
Triage & analytics

Synopsys Coverity
Deep code analysis,
High accuracy

OPEN SOURCE TOOLS

Semgrep
Fast pattern-based scanning,
Custom rules

CodeQL
Semantic code analysis,
Powerful queries

ESLint
Detects code issues,
Best practices

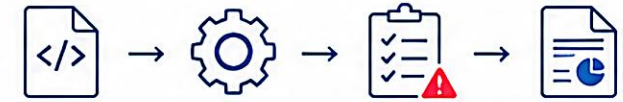
PMD
Finds bugs, vulnerabilities,
Code smells

LANGUAGE SUPPORT

Java · C# · C/C++ · Python
JavaScript · TypeScript · PHP · Go

... and more

HOW SAST WORKS



KEY CAPABILITIES

Static Code Analysis
Analyzes source code without executing it

Vulnerability Detection
OWASP Top 10, CWE, security hotspots

Compliance & Standards
OWASP, MISRA, CERT, PCI-DSS, etc.

Reporting & Dashboards
Actionable insights and trend analysis

CI/CD Integration
Shift-left security in DevSecOps

BENEFITS

Early Detection
Find vulnerabilities early in SDLC

Cost Effective
Fix issues early, reduce rework

Improve Security
Reduce risk of exploits

Boost Productivity
Automate reviews and quality gates

Ensure Compliance
Meet regulatory and policy requirements

DevSecOps Enablement
Integrate security into every pipeline

TRY IT YOURSELF — EXERCISE 3: FILL SAST PATH TODOS

Reinforces: tracing one real CRM request-to-sink path with your own hands, right after seeing what SAST tools find and how they detect it.

STEPS (notes/lab40-sast-todo-notes.md)

Step	What To Do
1 — Copy the Template	In notes, create a TODO block: Endpoint, Authz check, Sink (SQL/file/log), Customer fixture used, Risk if missing check.
2 — Fill for Customer Read	Fill the blanks for GET /api/customers/{id} using CUS-1001 (Amina). Authz must name the role/object-level check.
3 — Trace a Second Path	Repeat for a second path (e.g. customer search or update) using CUS-1002 (Ravi) as the fixture.
4 — Name the Risk	For each path, write one sentence on what happens if the authz check or parameterized query is missing.

SAST path TODO (fill in)

```
Endpoint: GET /api/customers/{id}
Authz check: ______ (role? object-level owner check?)
Sink (SQL/file/log): ______ (JPQL query? log statement?)
Customer fixture used: CUS-1001 (Amina Khan)
Risk if missing check: ______
```

KEY TAKEAWAY TRY IT NOW — Complete Exercise 3 before continuing: exercises/exercise-03-sast-todo-notes.md (workspace: examples/module-40-exercises).

WHAT IS DAST? (DYNAMIC APPLICATION SECURITY TESTING)

DAST is a black-box testing approach that evaluates a running application from the outside by simulating real-world attacks over HTTP/HTTPS.

DAST AT A GLANCE

- **Perspective —** external (black-box); no knowledge of internal code.
- **When —** after deployment, in QA, staging, or production-like environments.
- **How —** sends crafted requests and analyzes responses.
- **Finds —** runtime vulnerabilities exploitable via the app's interface.

COMMON VULNERABILITIES FOUND

- Injection (SQLi, NoSQLi, command injection)
- Authentication issues (weak login, session handling)
- Session management flaws (fixation, timeout, tokens)
- Authorization flaws (broken access control)
- Sensitive data exposure
- Business logic flaws (price tampering, workflow bypass)

DAST VS SAST

Aspect	SAST	DAST
Testing type	White-box	Black-box
When	Early (code)	After build
Input	Source code	Running app
Access needed	Yes	No (URL only)

BEST PRACTICES FOR DAST

Run Close to Production | **Use Authenticated Scans** | **Avoid Peak-Hour Scanning** | **Combine With SAST / SCA** | **Re-scan Regularly**

KEY TAKEAWAY DAST tests a deployed application in runtime to find vulnerabilities that attackers can exploit over the network -- injection flaws, auth issues, misconfigurations, and business logic flaws.

WHAT IS DAST?

DAST (Dynamic Application Security Testing) is a security testing method that analyzes a running application from the outside to identify vulnerabilities that could be exploited in real-world attacks.

HOW DAST WORKS

1. Target Application
A running application is identified as the test target.

2. Scanner Probes
DAST scanner sends requests (attacks) to the application.

3. Analyze Responses
The scanner analyzes the application's responses.

4. Report Findings
Vulnerabilities are identified and reported.

WHAT DAST DETECTS

- Injection flaws (e.g., SQL, NoSQL, OS Command)
- Cross-Site Scripting (XSS)
- Authentication & authorization issues
- Sensitive data exposure
- Business logic vulnerabilities
- Configuration & deployment issues

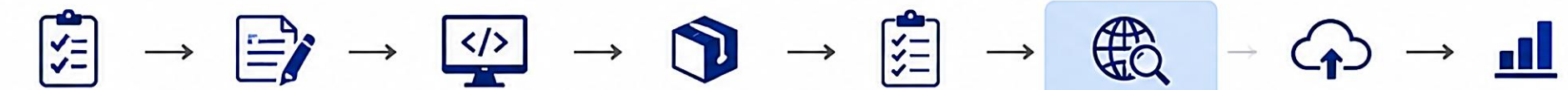
KEY CHARACTERISTICS

- Tests running application without access to source code
- Black-box testing approach
- Simulates real-world attacks
- Finds runtime & environment related issues
- Complements SAST and other testing methods

DAST vs SAST

SAST	Analyzes source code	White-box testing	Early in SDLC	Finds code-level issues
DAST	Analyzes running application	Black-box testing	Later in SDLC / Pre-production	Finds runtime & environment issues

WHERE DAST FITS IN THE SDLC



HOW DAST WORKS

DAST tools test a running application from the outside by simulating real-world attacks, then analyze the responses to identify vulnerabilities.

1. **Target Setup**
2. **Crawling & Discovery**
3. **Attack Simulation**
4. **Response Analysis**
5. **Vuln. Identification**
6. **Reporting & Retest**

EXAMPLE: SQL INJECTION (DAST)

```
GET /products?id=10
  -> 200 OK  Product: Laptop

GET /products?id=10 OR 1=1--
  -> 200 OK  All products returned!

// Behavior difference proves the parameter is vulnerable.
```

DAST OUTPUTS & RISK RATINGS

- **Outputs —** vulnerability details, proof of concept, attack vector, impact, remediation, retest status.
- **Risk Ratings —** Critical (immediate action), High (address soon), Medium (moderate), Low (monitor), Info.

COMMON DAST TESTING TECHNIQUES

Injection | **XSS** | **Auth Testing** | **Authorization** | **Business Logic** | **Config / Info**

KEY TAKEAWAY DAST behaves like an attacker: it sends crafted requests to the application, analyzes responses, and identifies vulnerabilities that are exploitable at runtime.

HOW DAST WORKS

1 TARGET IDENTIFICATION

The tester provides the target URL(s), login credentials (if needed), and scope.

Input:
URL, Auth, Scope

2 CRAWLING & DISCOVERY

DAST scanner crawls the application, identifies pages, links, forms, APIs and endpoints.

Output:
Site Map / Inventory

3 VULNERABILITY SCAN (ACTIVE)

The scanner sends crafted requests and payloads to detect security weaknesses.

Techniques:
Active Probing

4 RESPONSE ANALYSIS

Responses are analyzed to detect anomalies, errors, or unexpected behavior.

Analysis:
Compare & Detect

5 VULNERABILITY VERIFICATION

Potential issues are verified to confirm they are real and not false positives.

Validation:
Confirm & Validate

6 RISK ASSESSMENT

Each verified issue is prioritized based on severity, impact, and exploitability.

Output:
Risk Score / Rating

7 REPORTING & REMEDIATION

A detailed report is generated with findings and remediation guidance.

Output:
Report & Guidance

DAST SCANNING TECHNIQUES

Crawling

Discover URLs, pages, forms, parameters.

Active Probing

Send crafted requests and payloads.

Payload Injection

Test for SQLi, XSS, Command Injection, and more.

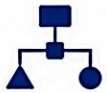
Authentication Handling

Handle logins, sessions, tokens, and roles.

Session Management

Maintain session state during scanning.

Business Logic Testing



Test workflows, logic flaws, and access control.

OUTPUT / RESULTS

- Vulnerabilities (e.g., SQLi, XSS, CSRF, etc.)
- Severity & Risk Ratings (Critical / High / Medium / Low)
- Affected URL / Parameter / Request
- Evidence (Request / Response)
- Remediation Guidance & References
- Exportable Reports (PDF, HTML, JSON, CSV)

KEY BENEFITS

- Finds runtime & real-world issues
- Tests from an attacker's perspective
- Detects issues SAST cannot find
- Supports compliance & security standards
- Reduces risk before deployment
- Improves application security posture

DAST ARCHITECTURE

DAST evaluates a running application from the outside by simulating real-world attacks to discover vulnerabilities that are exploitable at runtime.

1. **Target & Access**
2. **Scanner Engine**
3. **Attack & Payload Engine**
4. **Response Analysis**
5. **Reporting & Remediation**

DAST CAPABILITIES

- Black-box testing of running applications
- Crawling & endpoint discovery
- Automatic attack simulation
- Authenticated scanning
- Business logic testing
- Accurate vulnerability validation & retesting

EXAMPLE ATTACKS DETECTED

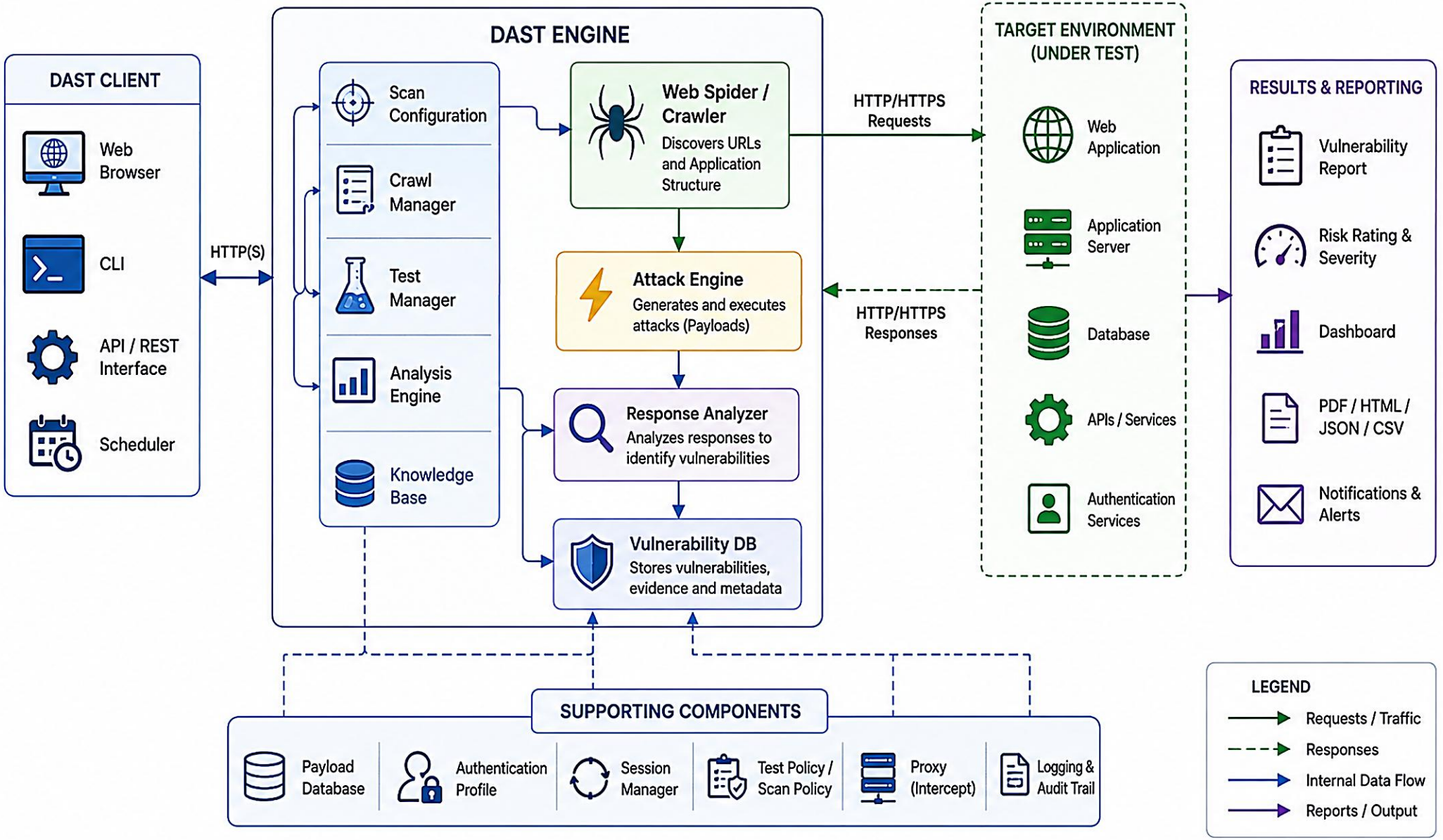
- SQL Injection
- Cross-Site Scripting (XSS)
- Authentication bypass
- Session management issues
- Broken access control (IDOR)
- Security misconfiguration

COMMON DEPLOYMENT MODES

- Cloud Service (SaaS)
- On-Premises (virtual/physical)
- Containerized (Docker/K8s)

KEY TAKEAWAY DAST finds vulnerabilities that exist in the running application and are exploitable by an attacker -- making it a critical layer in DevSecOps.

DAST ARCHITECTURE



COMMON VULNERABILITIES DETECTED BY DAST

DAST focuses on how the application behaves, not how the code is written -- finding issues an attacker can exploit through interfaces and business logic.

#	Vulnerability	Risk	DAST Detection Method	Remediation
1	SQL Injection	Critical	Injects payloads into parameters (GET/POST/cookies/headers) and analyzes responses.	Parameterized queries, least-privilege DB access.
2	Cross-Site Scripting	High	Injects scripts into inputs and checks for reflected, stored, or DOM-based XSS.	Output encoding, CSP, sanitize input.
3	Authentication Bypass	High	Tests weak credential handling, parameter tampering, session manipulation.	Enforce strong auth, MFA, secure sessions.
4	Session Management Issues	High	Checks session token entropy, predictability, timeout, reuse.	Regenerate session ID, secure/HttpOnly cookies.
5	Broken Access Control (IDOR)	High	Modifies object IDs/parameters and checks for unauthorized access.	Server-side access control, validate ownership.
6	Sensitive Data Exposure	Medium	Scans responses for PII, credit cards, stack traces, keys.	Mask sensitive data, disable detailed errors.
7	Security Misconfiguration	Medium	Checks common misconfigurations, default creds, verbose errors.	Harden configuration, disable debug mode.
8	Business Logic Flaws	Medium	Tests workflows, parameters, discounts, sequence of actions.	Enforce business rules server-side, rate limits.

KEY TAKEAWAY Not all findings reported by DAST tools are vulnerabilities. Combine DAST with SAST and manual testing to focus on what truly puts the application at risk.

COMMON VULNERABILITIES DETECTED BY DAST

1 SQL INJECTION

Occurs when untrusted input is used in SQL queries.

Example:

```
.../product?id=1' OR '1'='1
```

2 CROSS-SITE SCRIPTING (XSS)

Allows attackers to inject malicious scripts into web pages viewed by other users.

Example:

```
.../search?q=<script>alert(1)</script>
```

3 COMMAND INJECTION

Occurs when untrusted input is used to execute system commands.

Example:

```
.../ping?host=127.0.0.1; ls -la
```

4 CROSS-SITE REQUEST FORGERY (CSRF)

Tricks a user's browser into performing unwanted actions on a web application.

Example:

```
<img src=".../transfer?amount=1000">
```

5 INSECURE AUTHENTICATION

Weak login mechanisms or improper session management.

Example:

```
Logik without MFA or weak session cookies.
```

6 BROKEN ACCESS CONTROL

Users can access resources or functions without proper authorization.

Example:

```
.../admin/users (accessed without admin rights)
```

7 SENSITIVE DATA EXPOSURE

Sensitive data is transmitted or stored without proper encryption or protection.

Example:

```
Credit card number returned in response or via HTTP.
```

8 SECURITY MISCONFIGURATION

Applications or servers are configured insecurely, exposing unnecessary information or features.

Example:

```
Directory listing enabled or default credentials.
```

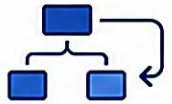
9 INSECURE DESERIALIZATION

Untrusted data is deserialized, leading to remote code execution or other attacks.

Example:

```
Vulnerable deserialization endpoint with crafted payload.
```

10 BUSINESS LOGIC FLAWS



Flaws in application logic that can be exploited to bypass rules or restrictions.

Example:

```
Applying discount multiple times or price manipulation.
```

11 INSECURE FILE UPLOAD

Allows upload of malicious files that can be executed on the server.

Example:

```
Uploading a PHP shell as image.jpg
```

12 PATH TRAVERSAL

Allows accessing files or directories outside the intended directory.

Example:

```
.../download?file=../../etc/passwd
```

13 INFORMATION DISCLOSURE

Exposes sensitive information about the application, server, or users.

Example:

```
Server errors revealing stack traces or config details.
```

14 UNSAFE CORS CONFIGURATION

Improper CORS settings allow unauthorized domains to access sensitive data.

Example:

```
Access-Control-Allow-Origin: *
```

15 CLICKJACKING

Tricks users into clicking hidden elements by overlaying transparent frames.

Example:

```
Missing X-Frame-Options or CSP.
```

WHY IT MATTERS

- Finds exploitable vulnerabilities in running applications
- Identifies issues attackers can leverage in the real world
- Improves security posture and reduces risk
- Supports compliance and security standards
- Protects users, data, and business reputation

DAST TOOLS OVERVIEW

DAST tools act like an attacker: they discover vulnerabilities in the running application, validate impact, and help fix issues before attackers can exploit them.

Tool	Key Features	Best For	Strengths
OWASP ZAP	Intercepting proxy, active & passive scanning, fuzzing & spider, scripting.	Developers, small teams, education, early testing.	Free & open source, active community.
Burp Suite Professional	Intercepting proxy, advanced active scanner, intruder (fuzzing), repeater.	Security testers, pen testers, enterprises.	Industry standard, powerful scanner & tools.
Acunetix	Automated scanning, vulnerability verification, WAF detection.	Enterprises, DevSecOps pipelines, compliance.	Accurate scanning, low false positives.
IBM AppScan	Automated scanning, advanced auth support, data & compliance checks.	Large enterprises, regulated industries.	Enterprise grade, detailed compliance reporting.
Rapid7 InsightAppSec	Accurate automated scans, IAST + DAST combined, risk scoring.	DevSecOps, large teams, shift-left security.	Low false positives, good DevOps integration.
Invicti (Netsparker)	Highly accurate scanning, proof-based validation, API & web scanning.	Enterprises, API security, high-assurance testing.	High accuracy, excellent API support.

WHEN TO USE DAST

- **After Deployment (QA / Staging)**
- **Before Release to Production**
- **For Compliance & Pen Testing**
- **In Combination With SAST & Manual**

KEY TAKEAWAY No single tool finds everything. Use the right tool, with the right configuration, at the right time.

DAST
TOOLS OVERVIEW

1 OWASP ZAP

- Open Source
- Web application security scanner
- Active & passive scanning
- Strong community support
- Integrates with CI/CD

2 Burp Suite

- Commercial tool (free community edition available)
- Advanced scanning & manual testing
- Extensive vulnerability coverage
- Powerful proxy & scanner
- Rich extender ecosystem

3 Acunetix

- Commercial tool
- High accuracy vulnerability scanning
- Deep scan with minimal false positives
- Supports CI/CD integration
- Compliance & reporting

4 Invicti (Netsparker)

- Commercial tool
- Proof-based scanning
- Accurate with low false positives
- Automatic vulnerability verification
- CI/CD & DevOps integration

5 Qualys Web Application Scanning

- Cloud-based solution
- Continuous web app monitoring
- Scales across large environments
- Easy deployment & management
- Detailed reporting & dashboards

6 Rapid7 InsightAppSec

- Commercial tool
- Automation-first scanning
- Integrates with CI/CD pipelines
- Comprehensive vulnerability coverage
- Risk-based prioritization

7 AppScan (HCL)

- Commercial tool
- Advanced DAST with IAST support
- High scalability
- Compliance & policy management
- Detailed analytics & reporting

8 PortSwigger DAST

- Cloud-based DAST
- Built by the creators of Burp Suite
- Easy to use & highly accurate
- CI/CD integration
- Developer-friendly reporting

9 SiteLock DAST

- Cloud-based scanning
- Automatic vulnerability detection
- Malware & blacklist monitoring
- Easy dashboards & alerts
- Designed for websites & web apps

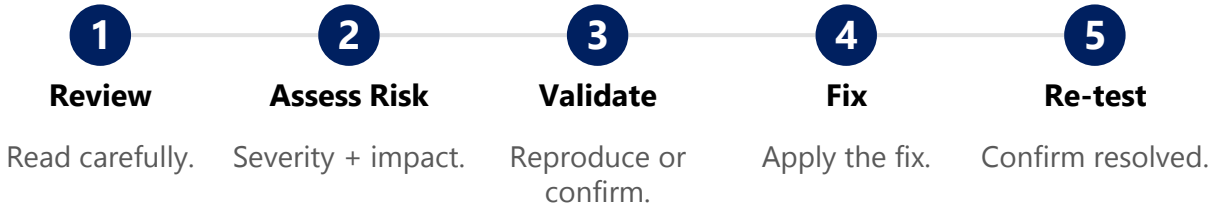
INTERPRETING SECURITY FINDINGS

Security findings help us understand what the issue is, where it exists, how it can be exploited, and what impact it can have -- so we can prioritize and fix it effectively.

UNDERSTAND EACH FIELD

- **Title —** short name of the vulnerability.
- **Description —** explains the issue in detail.
- **Location —** where the issue exists (file, line, endpoint, parameter).
- **Code Snippet / Evidence —** proof of the vulnerable code or response.
- **Impact —** what can go wrong if exploited.
- **Severity / CVSS —** risk score and business impact.
- **Recommendation —** what should be done to fix it.

HOW TO INTERPRET & ACT



TIPS FOR BETTER DECISIONS

Don't Trust Tool Severity Alone | **Trace Source-to-Sink Data Flows** | **Group Similar Findings** | **Combine SAST + DAST + Manual** | **Track Findings to Closure**

KEY TAKEAWAY Golden Rule: prioritize by Risk x Impact x Exploitability -- focus first on issues that are easy to exploit and have the highest business impact.

INTERPRETING SECURITY FINDINGS

1 REVIEW THE FINDING SUMMARY

- Read the finding title and description carefully
- Understand the vulnerability type
- Note the affected component / URL / function

2 ASSESS SEVERITY & RISK

- Check the severity score (CVSS or tool rating)
- Evaluate business impact
- Consider exploitability and ease of attack
- Prioritize based on risk and impact

3 UNDERSTAND THE EVIDENCE

- Review request/response or output
- Examine payload used and response
- Identify proof of vulnerability
- Reproduce the issue (if possible)

4 IDENTIFY THE ROOT CAUSE

- Determine why the vulnerability exists
- Map to OWASP Top 10 or relevant category
- Understand the flawed logic or configuration
- Identify the weak control

5 REVIEW REMEDIATION GUIDANCE

- Read the recommended fix provided
- Understand secure coding/configuration practices
- Check references or official guidelines
- Plan the remediation approach

6 VALIDATE & TRACK THE FINDING

- Verify the issue is fixed
- Re-test to confirm resolution
- Track status in your issue tracker
- Re-open if the issue persists

KEY CONSIDERATIONS

FOCUS ON RISK
Prioritize findings that pose the highest risk to your application and business.

CONTEXT MATTERS
Consider environment, user role, data sensitivity, and business logic.

AVOID FALSE POSITIVES
Validate the finding before taking action.

CONTINUOUS IMPROVEMENT
Use findings to improve security controls and prevent future issues.

COLLABORATE
Work with developers, security, and operations for effective resolution.

REMEMBER: A finding is only as valuable as the action you take on it. | Understand → Prioritize → Fix → Validate → Improve

VULNERABILITY SEVERITY CLASSIFICATION

Severity classification standardizes risk communication based on CVSS v3.1 and ensures critical issues get fixed first.

Severity	CVSS Range	What It Means	Example	Action
Critical	9.0 - 10.0	Exploitable easily with high impact; immediate threat to systems or data.	SQL injection (unauthenticated), RCE	Fix immediately (24-72 hrs)
High	7.0 - 8.9	Likely to be exploited; high business risk and potential damage.	Broken authentication, IDOR	Fix ASAP (1-2 weeks)
Medium	4.0 - 6.9	Possible to exploit with moderate effort; medium business impact.	Stored XSS, security misconfig	Plan to fix (1-4 weeks)
Low	0.1 - 3.9	Low risk or requires significant effort to exploit; limited impact.	Reflected XSS, missing headers	Fix when possible (1-3 mo.)
Info	0.0	Informational finding or best practice; no direct exploitable risk.	Tool/version disclosure	Track for future hardening

FACTORS THAT INFLUENCE SEVERITY

- **Exploitability —** how easy it is to exploit.
- **Data Sensitivity —** value and sensitivity of affected data.
- **Business Impact —** effect on business operations.
- **Exposure —** publicly accessible or internal.

SEVERITY VS. PRIORITY

- **Severity —** technical risk; how bad is it?
- **Priority —** business context; what to fix first?
- **Priority considers —** severity + business impact + exploitability + asset criticality.

KEY TAKEAWAY Golden Rule: focus on the vulnerabilities that pose the greatest risk to your business and customers -- not just the number of findings.

VULNERABILITY SEVERITY CLASSIFICATION

SEVERITY LEVEL	DESCRIPTION	CVSS SCORE RANGE (V3.1)	IMPACT	EXPLOITABILITY	RECOMMENDED ACTION
CRITICAL	Vulnerabilities that can be exploited easily and result in a significant impact to the system or business.	9.0 – 10.0 (High)	**Very High** Complete compromise of system, data loss, or business disruption.	**Easy** Exploited in the wild or easy to exploit with minimal effort.	**Immediate** Fix immediately. Apply patches or workarounds right away.
HIGH	Vulnerabilities that can be exploited and may lead to a significant impact.	7.0 – 8.9 (High)	**High** Major impact to confidentiality, integrity, or availability.	**Likely** Exploitation is likely or practical with moderate effort.	**High Priority** Fix as soon as possible.
MEDIUM	Vulnerabilities that are possible to exploit and could result in moderate impact.	4.0 – 6.9 (Medium)	**Medium** Limited impact to system or data.	**Possible** Exploitation is possible with some effort or specific conditions.	**Moderate Priority** Fix in a timely manner.
LOW	Vulnerabilities that have limited impact and are difficult to exploit.	0.1 – 3.9 (Low)	**Low** Minimal impact to system or data.	**Unlikely** Exploitation is unlikely or requires significant effort.	**Low Priority** Fix as part of regular maintenance.
INFORMATIONAL	Findings that do not represent a vulnerability but provide useful security information.	0.0 (None)	**Informational** No direct impact on confidentiality, integrity, or availability.	**Not Applicable** Not a vulnerability.	**No Action Required** Review and consider for best practices.

RISK ASSESSMENT METHODOLOGY

A risk assessment determines the likelihood and impact of a vulnerability being exploited so we can prioritize remediation based on business risk.

RISK ASSESSMENT PROCESS

- **1. Identify Assets —** critical applications, data, processes.
- **2. Identify Threats —** potential threat actors and attack vectors.
- **3. Identify Vulnerabilities —** use SAST/DAST/manual testing.
- **4. Determine Likelihood —** probability of exploitation.
- **5. Determine Impact —** potential business impact if exploited.
- **6. Calculate & Prioritize —** likelihood x impact, define treatment.

LIKELIHOOD SCALE

Level	Score	Criteria
Very High	5	Exploit code readily available
High	4	Public exploit available
Medium	3	Proof of concept available
Low	2	Difficult to exploit
Very Low	1	No known exploit

RISK TREATMENT OPTIONS

- **Mitigate —** reduce likelihood or impact (patch, code fix).
- **Transfer —** shift risk to a third party (insurance, SLA).
- **Accept —** accept with justification and monitoring.
- **Avoid —** remove the vulnerable component/feature.

RISK = LIKELIHOOD x IMPACT (RISK MATRIX)

1-4 Low Risk | **5-9 Moderate Risk** | **10-16 High Risk** | **17-25 Critical Risk**

KEY TAKEAWAY Golden Rule: focus on the risks that matter most to your business, not just the number of findings. Risk = Likelihood of Exploitation x Impact to the Organization.

RISK ASSESSMENT METHODOLOGY

1 ESTABLISH CONTEXT

- Define objectives
- Identify scope
- Understand business environment
- Identify assets
- Identify stakeholders

2 IDENTIFY RISKS

- Identify risk sources
- Identify events
- Identify affected assets
- Create risk register

3 ANALYZE RISKS

- Determine likelihood
- Determine impact
- Consider existing controls
- Calculate inherent risk

4 EVALUATE RISKS

- Compare against risk criteria
- Determine risk level
- Prioritize risks

5 TREAT RISKS

- Select treatment options
- Implement controls
- Develop action plan
- Assign owners

6 MONITOR & REVIEW

- Monitor risks & controls
- Review effectiveness
- Update risk register
- Communicate & report

RISK ASSESSMENT FLOW

RISK MATRIX (Likelihood vs Impact)

IMPACT	1 Rare	2 Unlikely	3 Possible	4 Likely	5 Almost Certain
5 Catastrophic	Medium	High	High	Extreme	Extreme
4 Major	Medium	Medium	High	High	Extreme
3 Moderate	Low	Medium	Medium	High	High
2 Minor	Low	Low	Medium	Medium	High
1 Insignificant	Low	Low	Low	Medium	Medium

LIKELIHOOD

- Extreme (Immediate Action)
- High (Priority Action)
- Medium (Planned Action)
- Low (Monitor)

RISK TREATMENT OPTIONS

AVOID
Eliminate the risk by stopping the activity

REDUCE / MITIGATE
Implement controls to reduce likelihood or impact

TRANSFER / SHARE
Shift the risk to a third party (e.g., insurance)

ACCEPT
Accept the risk with or without controls

MONITOR
Monitor the risk and reassess over time

KEY PRINCIPLES

- Structured and Systematic
- Based on Best Available Information
- Inclusive and Collaborative
- Proportional to Risk
- Dynamic and Iterative
- Communicated and Documented

FALSE POSITIVES VS REAL RISKS

Not all findings reported by SAST/DAST tools are vulnerabilities. Understanding the difference helps you focus on what truly puts your application and business at risk.

Aspect	False Positive (Not Exploitable)	Real Risk (Potentially Exploitable)
What it is	Tool flags code/behavior as vulnerable, but in context it is safe.	A real vulnerability that can be exploited by an attacker.
Why it happens	Tool limitations, incomplete context, compensating controls.	Weak code, missing controls, or insecure configurations.
Impact	No exploitable path; no business impact.	Data breach, system compromise, financial loss, reputational damage.
Action	Validate and close with justification; tune rules.	Fix, re-test, and verify; prioritize based on risk.
Goal	Reduce noise, save time, focus on real risks.	Eliminate exploitable weaknesses and reduce business risk.

INDICATORS OF REAL RISK

- Exploitable in test / proof of concept
- Impacts confidentiality, integrity, or availability
- No strong compensating control in place
- Reachable in production
- High severity + high business impact

MORE LIKELY FALSE POSITIVE

- No exploit path (proved safe)
- Protected by framework / parameterization
- Validated and documented
- Only theoretical risk
- Low or no business impact

KEY TAKEAWAY Golden Rule: treat every finding as a hypothesis -- verify, validate, then decide. Fix the real issues. Reduce the noise. Protect what matters most.

FALSE POSITIVES VS REAL RISKS

FALSE POSITIVES		REAL RISKS
DEFINITION A reported issue that is not actually a vulnerability in the given context.	DEFINITION	**DEFINITION** A genuine vulnerability that can be exploited to impact confidentiality, integrity, or availability.
CAUSE Scanner limitations, lack of context, safe configurations, or benign data.	CAUSE	**CAUSE** Weakness in code, configuration, or design that can be exploited under realistic conditions.
IMPACT Wastes time and resources, causes alert fatigue, may hide real issues.	IMPACT	**IMPACT** Can lead to data breach, system compromise, financial loss, or reputational damage.
VALIDATION Requires manual review, proof of exploitability, and context analysis to dismiss.	VALIDATION	**VALIDATION** Confirmed through testing, proof of concept (PoC), or exploit demonstration.
ACTION Document and close as false positive, improve scan tuning to reduce noise.	ACTION	**ACTION** Prioritize and remediate, then retest to confirm the issue is resolved.
EXAMPLE Scanner flags SQL Injection on a parameter that is not used in a query.	EXAMPLE	**EXAMPLE** SQL Injection in login form allows attacker to bypass authentication and access data.

KEY TAKEAWAY (FALSE POSITIVES)
Not all findings are real issues. Focus on verification and context to avoid chasing noise.

KEY TAKEAWAY (REAL RISKS)
Real vulnerabilities need prompt attention and remediation to reduce risk.

TRY IT YOURSELF — EXERCISE 4: SKETCH FINDINGS TRIAGE CSV

Reinforces: turning severity and false-positive-vs-real-risk judgment into a concrete triage ledger, before we move on to secure coding.

STEPS (notes/lab40-triage-csv-sketch.md)

Step	What To Do
1 — Define Columns	finding_id, cve, cvss, dependency, path, classification, owner, due_date, notes.
2 — Learn the Classifications	true_positive (confirm and fix, or accept with owner), false_positive (document rationale), accepted_risk (time-bounded, owned), fixed (re-scan evidence required).
3 — Sketch Two Sample Rows	Invent one true_positive on a transitive JAR and one false_positive with rationale (synthetic data only).
4 — Link to the CRM	Note how a true_positive on the API layer could affect agents opening CUS-1001's profile -- without claiming you fixed it yet.

Findings CSV header + sample rows

```
finding_id,source,package_or_location,cve_or_rule,cvss,classification,owner,due_date,notes
lab40-001,dependency-check,example:lib:1.2.3,CVE-2024-XXXX,7.5,needs_review,student,2026-07-21,transitive
lab40-002,sast,CustomerController#get,ACL-missing,,confirmed,student,2026-07-15,agent-a vs CUS-1002
```

KEY TAKEAWAY TRY IT NOW — Complete Exercise 4 before continuing: exercises/exercise-04-triage-csv-sketch.md (workspace: examples/module-40-exercises).

SECURE CODING BEST PRACTICES

Secure code is your first line of defense. Following secure coding practices reduces vulnerabilities, protects data and users, and builds trustworthy applications.

CORE PRINCIPLES

- **Defense in Depth —** use multiple layers of security.
- **Least Privilege —** minimum access required, nothing more.
- **Validate Everything —** never trust user input, client or server.
- **Fail Securely —** no stack traces or sensitive info in errors.
- **Secure by Default —** safe frameworks, libraries, configurations.

COMMON FIX: SQL INJECTION

```
// Bad
String q = "SELECT * FROM users"
  + " WHERE id='" + userId + "'";

// Good
String q = "SELECT * FROM users"
  + " WHERE id = ?";
PreparedStatement ps =
  conn.prepareStatement(q);
ps.setInt(1, userId);
```

SECURE CODING CHECKLIST

- Validate all inputs (type, length, format, range)
- Use parameterized queries / ORM
- Encode output based on context
- Use strong authentication & MFA
- Protect data (encryption, hashing)
- Avoid hardcoded secrets
- Keep dependencies up to date

SECURITY IN THE SDLC

1. **Requirements & Design** — Threat modeling.
2. **Development** — Secure coding practices.
3. **Testing** — SAST, DAST, code review.
4. **Deployment** — Secure configuration.
5. **Operations** — Monitor, patch, improve.

KEY TAKEAWAY Golden Rule: validate input, encode output, use least privilege, protect data, keep it simple. Secure today, safe tomorrow.

SECURE CODING BEST PRACTICES

1. INPUT VALIDATION

- Validate all user inputs on the server side.
- Use allowlists, not denylists.
- Enforce data type, length, format, and range checks.
- Sanitize data to prevent injection attacks.

2. OUTPUT ENCODING

- Encode output based on the context (HTML, JS, CSS, URL).
- Prevent Cross-Site Scripting (XSS) attacks.
- Use framework-provided encoding functions.

3. AUTHENTICATION & AUTHORIZATION

- Enforce strong authentication mechanisms.
- Implement role-based access control (RBAC).
- Apply least privilege principle.
- Never trust client-side authorization.

4. SECURE DATA STORAGE

- Encrypt sensitive data at rest.
- Store passwords using strong hashing (e.g., Argon2, bcrypt, PBKDF2).
- Protect encryption keys and secrets.

5. SECURE COMMUNICATION

- Use HTTPS/TLS for all communications.
- Validate SSL/TLS certificates.
- Prevent downgrade attacks.
- Use secure protocols and cipher suites.

6. PREVENT INJECTION ATTACKS

- Use parameterized queries or prepared statements.
- Avoid dynamic SQL.
- Use ORM/Query builders where possible.
- Validate and sanitize all inputs.

7. ERROR HANDLING & LOGGING

- Do not expose detailed error messages to users.
- Log errors securely with sufficient context.
- Avoid logging sensitive data (PII, credentials).

8. SESSION MANAGEMENT

- Use secure, HTTP-only, SameSite cookies.
- Regenerate session ID after login.
- Set appropriate session timeouts.
- Invalidate sessions on logout.

9. DEPENDENCY MANAGEMENT

- Use trusted and maintained libraries.
- Regularly scan for known vulnerabilities.
- Keep dependencies updated.
- Remove unused libraries.

10. SECURITY BY DESIGN

- Follow secure design principles.
- Model threats early.
- Apply defense in depth.
- Conduct regular security reviews and testing.

LEAST PRIVILEGE
Provide only the access necessary to perform a function.

CODE REVIEW
Perform regular code reviews to find security issues early.

AUTOMATED TESTING
Use SAST, DAST, and other tools in the SDLC pipeline.

DEVELOPER TRAINING
Keep developers updated on security threats and best practices.

CONTINUOUS IMPROVEMENT
Continuously monitor, test, and improve the security of your applications.

SECURITY TESTING IN CI/CD PIPELINES

Shift security left, automate early, and continuously protect. Integrate security testing throughout the CI/CD pipeline to deliver secure code to production faster and with confidence.

SECURITY TESTING TYPES

Type	What It Does
SAST	Analyzes source code for vulnerabilities without executing the application.
DAST	Tests a running application for vulnerabilities from an attacker's perspective.
SCA	Analyzes open-source dependencies for known vulnerabilities and license risks.
IAST / RASP	Analyzes the running application from inside, during tests or runtime.
Secrets Scanning	Detects hardcoded secrets, API keys, passwords, and tokens in code and configs.
Container & IaC Scanning	Finds misconfigurations and vulnerabilities in containers and infrastructure-as-code.

EXAMPLE SECURITY GATE POLICY

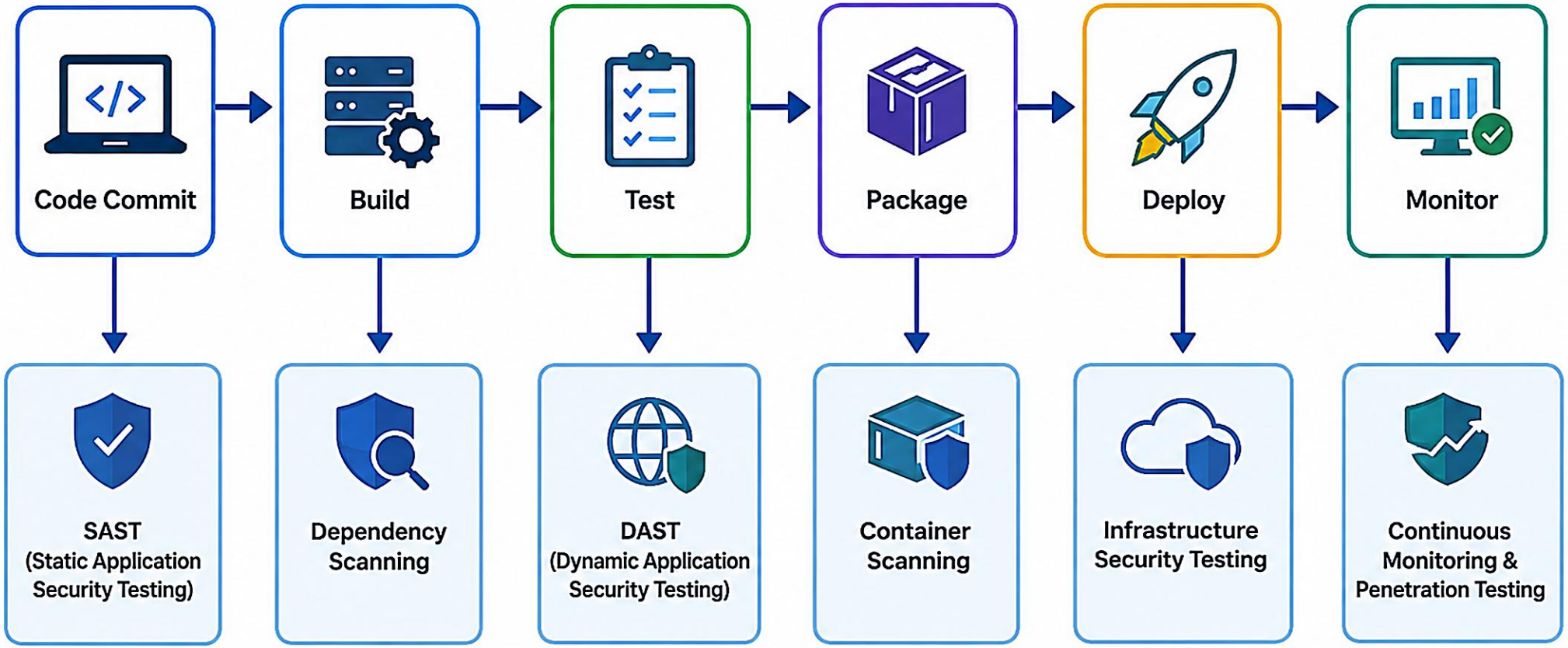
Severity	Action	Example
Critical	Fail pipeline	Any critical SAST/DAST/dependency issue
High	Fail or review	High issues > 0
Medium	Warn	Allowed up to threshold
Low/Info	Inform only	Best-practice suggestions

BEST PRACTICES

- **Automate on Every Commit**
- **Shift Security Left**
- **Use Multiple Testing Types**
- **Define & Enforce Quality Gates**
- **Keep Tools & Rules Current**
- **Track Metrics & Improve**

KEY TAKEAWAY Automate security. Enforce policies. Fix early. Deliver secure software -- every time.

Security Testing in CI/CD Pipelines



SECURITY GATES BEFORE PRODUCTION

Security gates are enforced quality and security checkpoints in the CI/CD pipeline. A release can proceed to production only when all mandatory criteria are met.

#	Gate	What It Checks	Threshold	Mandatory
1	SAST Gate	Source code vulnerabilities and unreviewed security hotspots.	Critical/High	Yes
2	SCA / Dependency Gate	Known critical/high vulnerabilities or risky licenses in open-source components.	Critical/High	Yes
3	Secrets Gate	Hardcoded secrets or credentials detected in code/config.	Any	Yes
4	DAST / IAST Gate	Critical/high runtime vulnerabilities (auth bypass, injection, XSS).	Critical/High	Yes
5	Container & IaC Gate	Container image and infrastructure-as-code risks.	Critical/High	Yes
6	Approval Gate	Human security & release approver sign-off.	N/A	Yes

ACTIONS ON GATE FAILURE

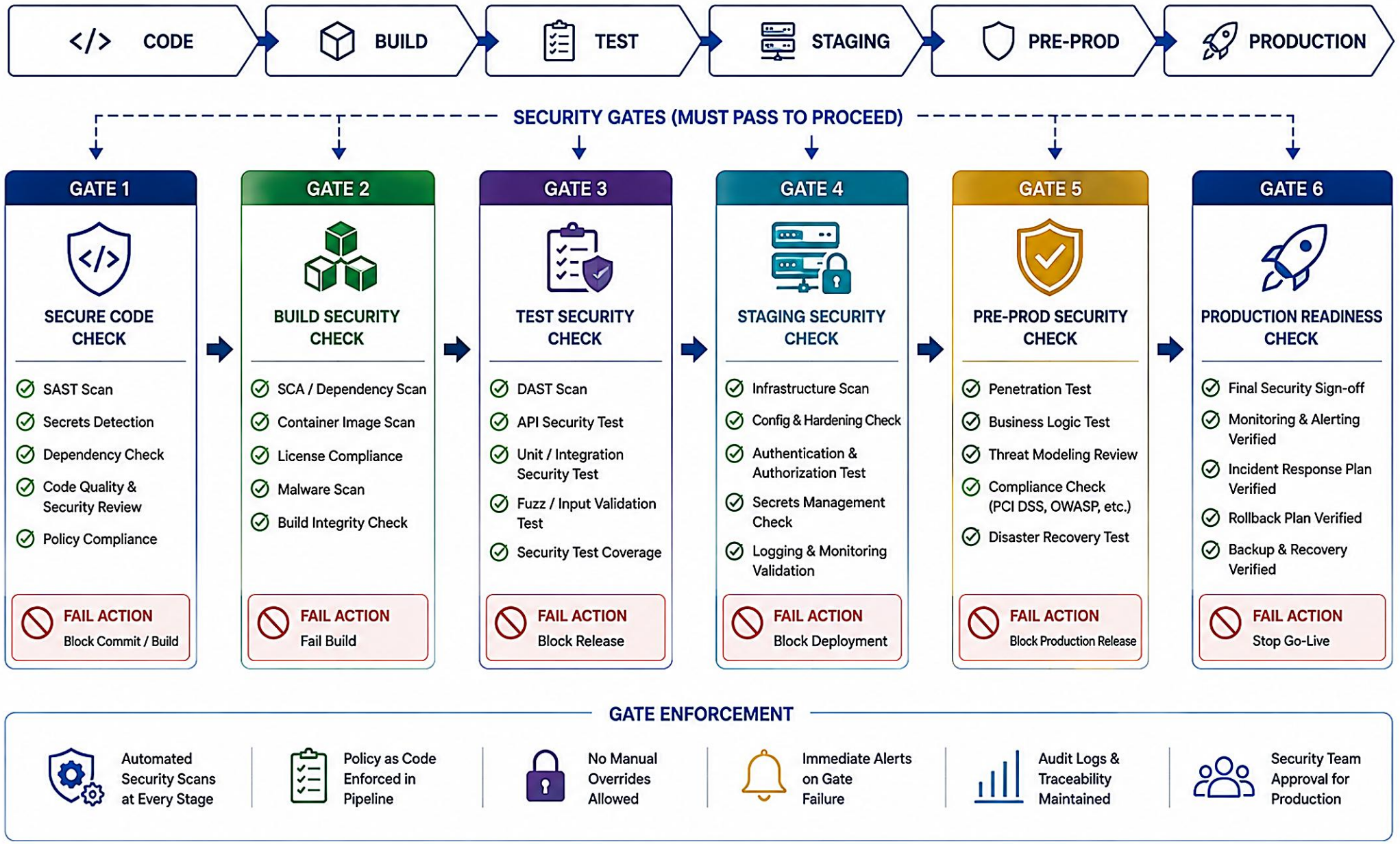
- **Pipeline is blocked —** the release is stopped; the artifact cannot move forward.
- **Notify & create ticket —** auto-create a ticket with scan details, assigned to the right team.
- **Fix, re-run, re-validate —** developers fix issues and all failed gates must pass again.

BEST PRACTICES

- Shift security left and test early
- Automate all security scans
- Make gates visible to everyone
- Combine automated checks with human judgment

KEY TAKEAWAY Golden Rule: if it doesn't pass the security gates, it doesn't go to production. Secure today. Protect tomorrow. Build trust.

SECURITY GATES BEFORE PRODUCTION



TRY IT YOURSELF — EXERCISE 5: OUTLINE SECURITY ASSESSMENT

Reinforces: turning severity, risk, and secure-coding judgment into the security-assessment document your Lab 40 gate must produce.

STEPS (notes/lab40-assessment-outline.md)

Step	What To Do
1 — List the Sections	Scope, tools, findings summary, remediations planned, residual risks, evidence index.
2 — Design the Residual-Risk Row	Each residual risk needs: risk, severity, owner, due date, and mitigating control.
3 — Draft the Evidence Index	Map each claim to an artifact path placeholder (Dependency-Check HTML, triage CSV, regression test name).
4 — Add the Scope-Honesty Line	State explicitly: this is a pre-lab outline only; full remediation and re-scan happen in Lab 40.

security-assessment.md outline

```
# Security Assessment - Lab 40 CRM
## Scope and assets
## Method and tool versions
## Dependency findings (before / after)
## Manual SAST findings
## Remediation summary (lab40-00x)
## Residual risks (owner, due date)
## Facts vs assumptions
## Reproduce commands
```

KEY TAKEAWAY TRY IT NOW — Complete Exercise 5 before continuing: exercises/exercise-05-assessment-outline.md (workspace: examples/module-40-exercises).

TRY IT YOURSELF — EXERCISE 6: GO/NO-GO READINESS

The capstone pre-lab exercise: confirm every prior exercise is genuinely done before opening the real, graded Lab 40.

STEPS (notes/lab40-gate-go-nogo.md)

Step	What To Do
1 — Draft Five Questions	Is every High/Critical CVE owned? Are there secrets in Git? Is there an authz negative test? Is the suppression policy followed? Does verify stay green?
2 — Check Leadership's Rule	No ship on raw scanner volume; no silent suppressions; no secrets committed -- ever.
3 — Tie Each Question to the CRM	For each question, write one line on the impact to agents serving Amina (CUS-1001) and Ravi (CUS-1002).
4 — Self-Mark Pass / Fail	Pass only if Exercises 1-5's notes files genuinely exist and are filled in -- not rubber-stamped.

KEY TAKEAWAY TRY IT NOW — Complete Exercise 6 before continuing: exercises/exercise-06-gate-go-nogo.md (workspace: examples/module-40-exercises). Then open the Lab 40 OS guide.

LAB OVERVIEW -- APPLICATION SECURITY TESTING FOR THE CRM

Lab 40: OWASP Dependency-Check (SCA), focused manual SAST, and one verified authorization fix -- a defensible security gate for the Northstar CRM before containers.

BUSINESS SCENARIO

- Leadership freezes a release gate before Lab 41's containers: scanners alone are not enough -- every confirmed finding needs severity rationale, evidence, and either a fix, a time-bounded acceptance with owner, or a documented false positive.
- No ship on raw scanner volume. No merge that commits secrets, disables the scan gate, or leaves a confirmed High without owner and due date.
- You own that gate for the CRM backend serving Amina (CUS-1001, ACTIVE) and Ravi (CUS-1002, PROSPECT), and agent role boundaries.

LEARNING OBJECTIVES

- Map CRM attack surfaces to OWASP-aligned risks.
- Run OWASP Dependency-Check via Maven with HTML/JSON output.
- Interpret CVE, CVSS, CPE, and transitive dependency paths.
- Perform focused manual SAST on injection, authz, and secrets.
- Triage false positives and accepted risks with owners and expiry.
- Reproduce one confirmed issue with an automated test, then remediate and re-scan.

WHAT YOU BUILD

- lab40-crm branched from lab39-crm; a Dependency-Check Maven profile (HTML+JSON, CVSS fail threshold); a triaged findings CSV; manual SAST notes on request-to-sink and object-level authorization paths; a failing-then-passing ObjectOwnershipSecurityTest; the smallest safe remediation; and docs/security-assessment.md with residual risks owned.

KEY TAKEAWAY Fixtures CUS-1001 (Amina Khan, ACTIVE) and CUS-1002 (Ravi Singh, PROSPECT) anchor every example; correlation ID lab-request-001 appears on every security-relevant error; use synthetic emails only, never real customer data.

LAB TASKS AND DELIVERABLES

lab40-crm -- implementation steps, deliverables, and the evaluation rubric.

#	Implementation Step
1	Branch lab40-crm from lab39-crm; confirm baseline mvnw clean verify.
2	Establish scope and a threat checklist; define the findings-CSV columns.
3	Add the OWASP Dependency-Check Maven profile (HTML+JSON, CVSS fail threshold, suppression file).
4	Run the scan and preserve command + tool-version evidence.
5	Triage findings by exploitability, reachability, and CVSS -- not just count them.
6	Perform focused manual SAST on request-to-sink paths and object-level authorization.
7	Reproduce one confirmed issue with a failing ObjectOwnershipSecurityTest.
8	Remediate with the smallest safe, root-cause fix -- never disable the gate.
9	Re-scan, regression-test, and write docs/security-assessment.md.

Object-ownership regression test

```
@Test @WithMockUser(username = "agent-a", roles = "AGENT")
void agentCannotReadAnotherAgentsCustomer() throws Exception {
  mvc.perform(get("/api/customers/{id}", otherAgentsCustomerId)
      .header("X-Correlation-Id", "lab-request-001"))
     .andExpect(status().isForbidden());
}
```

EXPECTED DELIVERABLES

- Threat checklist + OWASP mapping notes.
- Dependency-Check profile, reports (sanitized), triage CSV.
- Focused SAST notes with code locations.
- Security regression test + remediation evidence.
- Before/after scan comparison for the fixed finding.
- docs/security-assessment.md with residual risks owned.

RUBRIC (100 MARKS)

- Environment/Structure 10 · Core Impl 30 · Integration/Config 15 · Failure Handling 15 · Verification 10 · Security/Prod Awareness 10 · Docs 10

KEY TAKEAWAY Mass suppressions without expiry lose security marks. Disabling the gate or @Disabled on the ownership test is an honor violation. An assessment without commands/versions loses documentation marks.

MODULE 40 SUMMARY

In this module, you learned how to test applications for security vulnerabilities, assess risk, and ensure secure delivery through automated and manual testing in the SDLC and CI/CD pipelines.

KEY CONCEPTS COVERED

SSDLC & OWASP Top 10

Integrated security into every SDLC phase and learned the OWASP Top 10 risks.

SAST & DAST

Applied static and dynamic testing to find code-level and runtime vulnerabilities.

Risk & Triage

Classified severity, assessed risk, and separated false positives from real risk.

Secure Coding

Applied secure coding principles: validate input, encode output, least privilege.

CI/CD Security Gates

Integrated SAST/DAST/SCA into pipelines and enforced gates before production.

Hands-on Lab

Built lab40-crm: Dependency-Check, focused SAST, and one verified authz fix.

MODULE FLOW RECAP

1. Code
2. SAST
3. DAST
4. Risk & Triage
5. Lab: lab40-crm

KEY TAKEAWAY Security testing is not a one-time activity -- it's a continuous process that helps us find issues early, prioritize what matters, and deliver secure, resilient applications.

KNOWLEDGE CHECK (1 OF 2)

Test your understanding of application security testing goals, SAST vs. DAST, and common web vulnerabilities.

1. What is the primary goal of application security testing?

A. To improve application performance

B. To find security vulnerabilities and reduce risk

C. To increase code complexity

D. To ensure the UI looks good

2. Which testing type analyzes source code without executing the application?

A. DAST

B. SAST

C. IAST

D. Fuzz Testing

3. Which tool is commonly used for dynamic application security testing (DAST)?

A. SonarQube

B. OWASP ZAP

C. Dependency-Check

D. VS Code

4. What does SCA stand for?

A. Source Code Analysis

B. Static Code Analysis

C. Software Composition Analysis

D. Secure Configuration Assessment

KEY TAKEAWAY Application security testing exists to find vulnerabilities and reduce risk; SAST analyzes code without executing it; OWASP ZAP is a common DAST tool; SCA stands for Software Composition Analysis -- exactly what OWASP Dependency-Check performs.

KNOWLEDGE CHECK (2 OF 2)

Four more questions on risk scoring, common web vulnerabilities, handling SAST false positives, and matching vulnerability types.

5. Which of the following is an example of a common web application vulnerability?

A. SQL Injection

B. High CPU usage

C. Large file upload (non-malicious)

D. Memory optimization

6. Risk is best calculated using which formula?

A. Risk = Impact + Effort

B. Risk = Likelihood + Impact

C. Risk = Likelihood x Impact

D. Risk = Impact / Likelihood

7. A SAST scan reports a 'hardcoded password' in a test file that is verified to never run in production. What should you do?

A. Ignore it without any action

B. Mark it as False Positive with justification and close

C. Delete the test file without reviewing

D. Commit the change without tracking

8. Which vulnerability type best matches: "Accessing other users' data by manipulating the object ID"?

A. SQL Injection

B. Cross-Site Scripting (XSS)

C. Insecure Direct Object Reference (IDOR)

D. Security Misconfiguration

KEY TAKEAWAY SQL Injection is a classic web vulnerability; Risk = Likelihood x Impact; a verified false positive should be closed WITH written justification, never silently ignored or deleted; IDOR means accessing other users' data by manipulating an object ID -- exactly the object-level authorization gap Lab 40's own regression test proves fixed.

"Security is not a phase, it's a mindset and a continuous responsibility."

MODULE 40 COMPLETE

Application Security Testing

You can now identify, test, assess, and fix application security vulnerabilities using OWASP Top 10, SAST, DAST, risk-based triage, secure coding, and CI/CD security gates.